Informe – Desafío II

Diseño e Implementación de la Red Social Ude@Book en C++

Sofía Arboleda Zapata
Jose Manuel Arias Toro
Universidad de Antioquia

16 de octubre de 2026

Índice

1. Contextualización y análisis del problema **3**

2. Evolución de la solución y decisiones de diseño **3**

2.1. Mensajes centralizados en `UdeABook` . . . 3
2.2. Amistades por índices y matriz triangular de bits . . . 4
2.3. Solicitudes pendientes en una segunda matriz de bits . . . 4
2.4. Carga directa de álbumes y fotos . . . 4
2.5. Orden de los mensajes por fecha . . . 5
2.6. Resumen de búsquedas y crecimiento de las estructuras . . . 5

3. Diseño y arquitectura de la solución **5**

3.1. Descripción de las clases . . . 5
3.2. Relaciones entre las clases . . . 6
3.3. Sobrecarga de operadores y constructores de copia . . . 6
3.4. Diagramas de clases . . . 6

4. Formato de los archivos de datos **8**

5. Algoritmos esenciales **9**

5.1. Carga y guardado de datos . . . 9
5.2. Inicio de sesión y fotos de amigos . . . 9
5.3. Solicitudes de amistad . . . 9
5.4. Sugerencias de amistad . . . 9
5.5. Mensajes . . . 10
5.6. Perfil de un amigo y carrusel . . . 10

6. Medición del consumo de recursos **10**

7. Problemas de desarrollo **11**

7.1. Aprendizajes 12

1. Contextualización y análisis del problema

Estamos construyendo una versión pequeña y sencilla de una red social. El programa carga los usuarios desde archivos, permite iniciar sesión, tener amigos, enviar y aceptar solicitudes de amistad, ver álbumes y fotos, dar *likes*, recibir sugerencias de amistad y enviar mensajes. Además, cada vez que termina una funcionalidad debe mostrar cuánto trabajo hizo (iteraciones) y cuánta memoria está usando.

El enunciado pone restricciones que condicionan todo el diseño: no se pueden usar contenedores de la STL ni `struct`, no se puede usar herencia, las estructuras de datos deben ser propias y con memoria dinámica, y el almacenamiento permanente no puede usarse como área de trabajo. A esto se suma que la eficiencia es parte fundamental de la evaluación.

La restricción que más nos costó fue trabajar sin la STL. Normalmente un `vector` resolvería con facilidad las listas que crecen, pero aquí tuvimos que pensar cómo crecer sin crear arreglos cada vez más grandes y copiar todo su contenido. De esa necesidad salió el uso de bloques enlazados, que aparece varias veces en el informe.

Lo más difícil de entender al leer el enunciado fue pasar de lo que se pedía a una estructura clara de memoria: quién es dueño de cada objeto, dónde guardar las amistades y los mensajes, y cómo medir iteraciones y memoria. También entendimos que no bastaba con que el programa funcionara: había que poder justificar cómo se almacenaba cada dato.

2. Evolución de la solución y decisiones de diseño

El preinforme planteó una idea inicial y, al implementarla, algunas decisiones cambiaron. Esta sección cuenta qué cambió y por qué.

2.1. Mensajes centralizados en `UdeABook`

Un mensaje pertenece a dos usuarios a la vez: el emisor lo tiene en sus enviados y el receptor en sus recibidos. En el preinforme propusimos que el emisor fuera el dueño del mensaje, pero esto complicaba el manejo de memoria: solo el emisor podía hacer `delete`, el receptor nunca debía hacerlo y el mensaje tenía que seguir existiendo mientras el receptor lo necesitara.

Finalmente centralizamos los mensajes en `UdeABook`, que crea, almacena y destruye todos los objetos `Mensaje`. Cada mensaje existe una sola vez en memoria. Los usuarios ya no guardan copias ni índices de los mensajes, sino punteros no propietarios hacia esos mismos objetos. Esto es seguro porque `UdeABook` es el único responsable de liberar los mensajes.

Para que un mismo mensaje pueda pertenecer al mismo tiempo a la lista de enviados de su emisor y a la lista de recibidos de su receptor, la clase `Mensaje` tiene dos enlaces independientes: `siguienteEnviado` y `siguienteRecibido`. Por su parte, `Usuario` conserva los punteros al primero y al último mensaje de cada lista, además de sus cantidades. De esta forma agregar un mensaje al final de la lista de enviados o recibidos no exige recorrer los mensajes anteriores.

El almacenamiento central continúa organizado en bloques enlazados de 64 objetos `Mensaje`. Elegimos 64 como una decisión práctica: no es tan pequeño como para crear bloques constantemente ni tan grande como para reservar demasiadas posiciones vacías de una sola vez. Cuando un bloque se llena se reserva uno nuevo y se enlaza, sin copiar los mensajes anteriores.

2.2. Amistades por índices y matriz triangular de bits

La idea principal del preinforme se mantuvo: representar quién es amigo de quién con una matriz de adyacencia, donde cada usuario se identifica por su posición en el arreglo general de usuarios. Sin embargo, la representación se optimizó durante la implementación.

En el preinforme planteamos una celda de un byte por pareja. Posteriormente pasamos a una matriz completa de bits, almacenada en un arreglo de `unsigned char`, con un bit por posición `[fila][columna]`. Aunque así ya ahorrábamos memoria, esa versión seguía repitiendo cada amistad: los bits `[a][b]` y `[b][a]` representaban el mismo vínculo.

En la versión definitiva aprovechamos que la amistad es simétrica y guardamos una sola mitad de la matriz, sin su diagonal. Una persona no puede ser amiga de sí misma, por lo que tampoco es necesario reservar esos bits. La estructura sigue siendo un arreglo de una dimensión, pero ahora solo necesita

$$\frac{N(N-1)}{2} \text{ bits},$$

donde N es la cantidad de usuarios. Para ubicar una pareja de usuarios (a, b), la función `calcularIndiceTriangular` toma $f = \text{máx}(a, b)$ y $c = \text{mín}(a, b)$ y calcula

$$\text{indice} = \frac{f(f-1)}{2} + c.$$

Después ubica el byte con `indice / 8` y el bit con `indice % 8`. Como ambos órdenes llevan al mismo índice, `sonAmigos(a,b)` y `sonAmigos(b,a)` consultan el mismo bit. Para establecer una amistad basta con encenderlo una sola vez.

Con 1000 usuarios, la matriz completa anterior ocupaba 125 000 bytes. La triangular ocupa $\lceil 499\,500/8 \rceil = 62\,438$ bytes, de modo que ahorramos 62 562 bytes, aproximadamente un 50 % de la memoria de amistades.

Decidimos definitivamente no crear una clase `Amigos`. Las funciones para matrices completas y triangulares quedaron en el módulo `MatrizBits`; ambas matrices pertenecen a `UdeABook`. Además, `Usuario` no guarda una lista de amigos: la información se consulta en la matriz triangular central.

2.3. Solicitudes pendientes en una segunda matriz de bits

En la implementación final las solicitudes de amistad tampoco se guardan en listas dentro de cada usuario. `UdeABook` mantiene una segunda matriz de bits, pero esta sí es completa y de tamaño $N \times N$. El bit `[emisor][receptor]` indica si existe una solicitud pendiente del emisor hacia el receptor.

A diferencia de la amistad, esta relación es dirigida: una solicitud de 3 hacia 8 no significa que exista una solicitud de 8 hacia 3. Por eso aquí no podemos aplicar la misma compresión triangular. Con 1000 usuarios, las solicitudes ocupan 125 000 bytes, mientras que las amistades ocupan 62 438. En conjunto, ambas matrices necesitan 187 438 bytes, frente a los 250 000 bytes de dos matrices completas.

2.4. Carga directa de álbumes y fotos

Otra mejora fue evitar copias temporales durante la carga. Al leer `usuarios.txt` ya conocemos cuántos álbumes tiene cada usuario, por lo que se reserva directamente su arreglo definitivo. Después, al leer `albumes.txt`, cada álbum reserva exactamente la cantidad de fotos indicada y estas se llenan en su posición final con los datos de `fotos.txt`. Así no es necesario crear arreglos intermedios para después copiarlos completos.

2.5. Orden de los mensajes por fecha

El enunciado pide mostrar los mensajes ordenados por fecha. En la solución final ya no dependemos de que `mensajes.txt` venga ordenado ni de que el orden físico de almacenamiento coincida con el cronológico.

Los mensajes se conservan en orden de llegada dentro de las listas enlazadas de cada usuario. Cuando se necesitan mostrar los no leídos o los que pertenecen a un rango de fechas, se recorre la lista de mensajes recibidos, se guardan temporalmente punteros a los mensajes que cumplen la condición y esos punteros se ordenan por fecha mediante ordenamiento por inserción. No se copian los objetos `Mensaje`; solamente se reordenan referencias temporales a ellos.

El ordenamiento es estable: si dos mensajes tienen la misma fecha, conservan entre ellos el orden de llegada. Cuando un usuario envía un mensaje nuevo se usa `Fecha::hoy()`, que toma la fecha actual ajustada a la hora de Colombia y guarda día, mes y año.

2.6. Resumen de búsquedas y crecimiento de las estructuras

No usamos un mismo algoritmo para todas las búsquedas. Los usuarios se localizan por nombre mediante búsqueda secuencial en el arreglo general; consultar una amistad o una solicitud concreta se hace por acceso directo a un bit. Para recuperar mensajes recibidos recorremos exclusivamente la lista enlazada del destinatario, sin revisar todos los mensajes del sistema.

El arreglo de usuarios y los arreglos de álbumes y fotos se reservan con tamaños conocidos durante la carga. Los arreglos auxiliares de sugerencias y consultas se reservan temporalmente y luego se liberan. La estructura que debe crecer durante el uso normal es el almacenamiento de mensajes: cuando se llena un bloque de 64 se agrega otro, sin copiar los mensajes anteriores. Así se cumple la decisión de evitar redimensionar y copiar.

3. Diseño y arquitectura de la solución

3.1. Descripción de las clases

- `Fecha`: maneja día, mes y año, valida que la fecha exista y permite compararlas, algo clave para ordenar y buscar mensajes. También entrega la fecha actual.
- `Foto`: guarda la fecha de captura, el tamaño, la ruta del archivo y la cantidad de *likes*.
- `Album`: funciona como una carpeta con nombre, ruta, privacidad y un arreglo de fotos. Decide si un visitante puede verlo según su privacidad.
- `Mensaje`: guarda el texto, los índices del emisor y del receptor, la fecha y si ya fue leído. Además contiene enlaces separados para pertenecer a la lista de enviados, a la lista de recibidos y al almacenamiento por bloques.
- `Usuario`: agrupa la información personal, sus álbumes y los punteros al primero y al último mensaje enviado y recibido. Estos punteros no hacen al usuario dueño de los mensajes.
- `Metricas`: cuenta las iteraciones, registra la memoria y reporta el uso de componentes externos empleados por cada funcionalidad.
- `UdeABook`: es la clase principal. Guarda el arreglo de usuarios, las matrices de amistades y solicitudes y el almacenamiento central de mensajes; lee y guarda los archivos, controla el inicio de sesión y ejecuta las funcionalidades que el usuario elige en el menú.

3.2. Relaciones entre las clases

`UdeABook` es dueño del arreglo de `Usuario`, de las dos matrices de bits y del almacenamiento central de `Mensaje`. `Usuario` es dueño de sus `Album`, pero solamente mantiene asociaciones no propietarias con los mensajes mediante punteros. `Album` es dueño de sus `Foto`. Tanto `Foto` como `Mensaje` contienen una `Fecha`.

`MatrizBits` no es una clase, sino un módulo de funciones que trabaja sobre arreglos de `unsigned char`. Se utiliza tanto para `matrizAmistades` como para `matrizSolicitudes`, ambas pertenecientes a `UdeABook`.

3.3. Sobrecarga de operadores y constructores de copia

En `Fecha` sobrecargamos los operadores `<`, `==` y `<=` para comparar fechas, además del operador de asignación. También tienen operador de asignación `Foto`, `Album` y `Usuario`, con lo que superamos el mínimo de dos operadores que pide el enunciado.

`Fecha`, `Foto`, `Album` y `Usuario` tienen constructor de copia. `Foto` y `Album` realizan copia profunda de la memoria que poseen. `Usuario` también copia de forma independiente sus datos personales y álbumes, pero sus punteros a mensajes se conservan como referencias no propietarias a los mismos objetos, porque los mensajes pertenecen a `UdeABook`.

En cambio, `Mensaje` y `UdeABook` tienen la copia deshabilitada con `= delete`. No necesitamos duplicarlos y una copia superficial sería peligrosa porque ambos participan en estructuras de memoria centralizadas cuya propiedad debe permanecer única.

3.4. Diagramas de clases

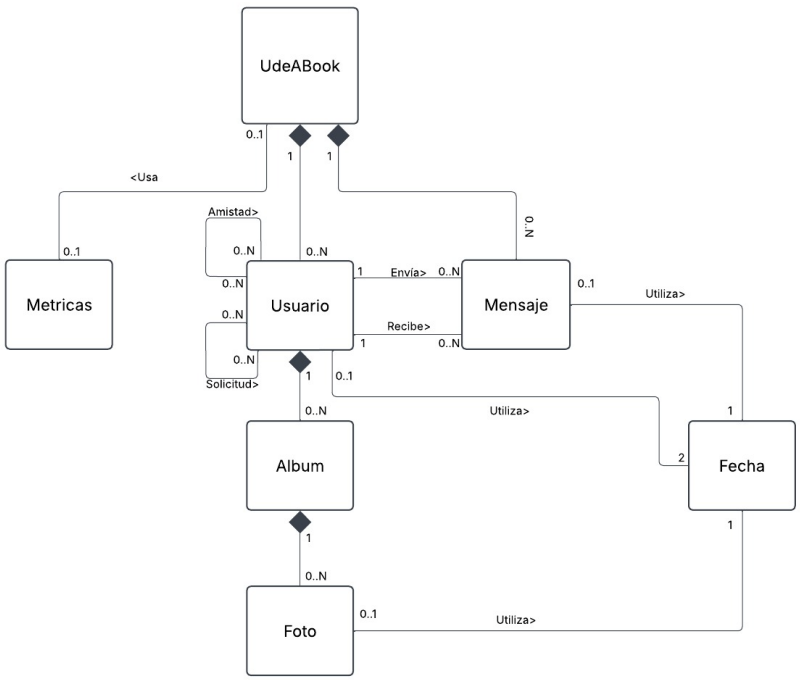


Figura 1: Diagrama de clases resumen

La Figura 1 muestra las relaciones principales del sistema. El detalle de atributos y métodos de cada clase se encuentra en las Figuras 2 a 4.

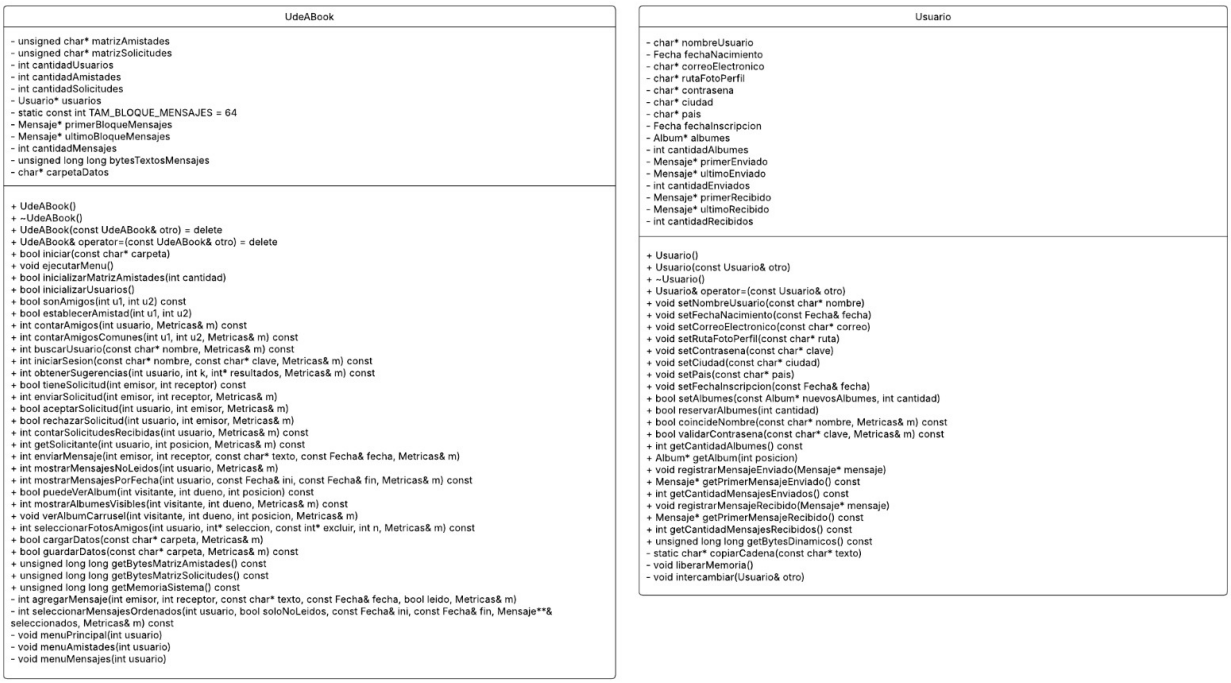


Figura 2: Clases expandidas (1)



Figura 3: Clases expandidas (2)

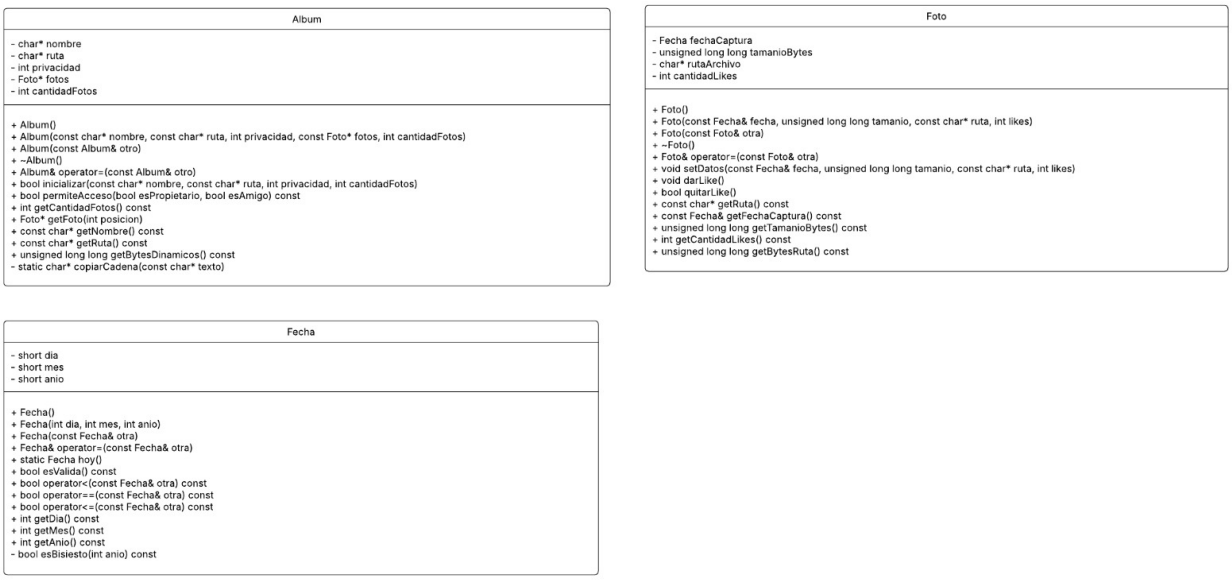


Figura 4: Clases expandidas (3)

4. Formato de los archivos de datos

Usamos seis archivos de texto, construidos con ayuda de IA (replico plantilla que le dimos) ya que no queriamos generar aleatoriamente nombres de usuarios, de album o de foto que no fueran muy dicientes, de hecho pusimos nombres de personas y personajes animados o superheroes y demas porque queriamos que visualmente fuera agradable al momento de ejecutar este desafio y que se viera realista. Todos usan la barra invertida (\) como separador, y su primera línea indica cuántos registros contiene el archivo. Los usuarios, álbumes y fotos se asocian mediante índices, no por nombre. Las solicitudes también se almacenan en los archivos mediante índices de emisor y receptor, aunque en memoria se representan mediante una matriz de bits.

- `usuarios.txt`: índice, nombre de usuario, día, mes y año de nacimiento, correo, ruta de la foto de perfil, contraseña, ciudad, país, día, mes y año de inscripción, y cantidad de álbumes.
- `albumes.txt`: índice del usuario, posición del álbum dentro de ese usuario, nombre, ruta, privacidad y cantidad de fotos.
- `fotos.txt`: índice del usuario, posición del álbum, posición de la foto, día, mes y año de captura, tamaño en bytes, ruta y cantidad de *likes*.
- `amistades.txt`: índice de los dos usuarios que son amigos.
- `solicitudes.txt`: índice del emisor y del receptor de cada solicitud pendiente.
- `mensajes.txt`: índice del mensaje, emisor, receptor, día, mes y año, estado de lectura y texto.

5. Algoritmos esenciales

5.1. Carga y guardado de datos

Los archivos se cargan en este orden: primero los usuarios, después los álbumes y las fotos, luego las amistades, las solicitudes y por último los mensajes. El orden importa: hay que conocer primero cuántos usuarios hay para crear el arreglo de usuarios, la matriz triangular de amistades y la matriz completa de solicitudes; los álbumes y fotos necesitan que su usuario ya exista; las solicitudes necesitan usuarios y revisar las amistades; y los mensajes necesitan que emisor y receptor ya existan para enlazar el mismo objeto a las listas de ambos.

Si una línea de usuarios, álbumes, fotos o mensajes tiene un formato incorrecto, la carga se detiene y muestra dónde ocurrió el error.

5.2. Inicio de sesión y fotos de amigos

Para iniciar sesión, el programa busca el nombre de usuario; si no existe, devuelve -1. Si existe, compara la contraseña y, si no coincide, también devuelve -1. Si todo es correcto, devuelve el índice del usuario.

Al ingresar se muestran tres fotos de perfil de amigos elegidos al azar, y no todos tienen la misma probabilidad: quienes tienen 10 o menos amigos en común con el usuario tienen prioridad simple, quienes tienen entre 11 y 20 tienen 2,5 veces más, y quienes tienen más de 20 tienen 3,5 veces más. Para evitar números decimales usamos pesos enteros equivalentes (2, 5 y 7, que son 1, 2,5 y 3,5 multiplicados por 2). El algoritmo suma todos los pesos disponibles, genera un número aleatorio entre 0 y el total menos uno, y recorre los candidatos restando sus pesos hasta encontrar a cuál le corresponde ese número.

Cuando un amigo ya fue elegido, su peso pasa a 0 para que no pueda volver a salir dentro del mismo grupo. Para la actualización, que solo puede hacerse una vez (un indicador del menú pasa a verdadero tras usar la opción `N`), primero se marcan los tres amigos anteriores como excluidos y se intenta elegir otros. Si no hay suficientes amigos distintos, se hace una segunda ronda en la que sí se permite repetir alguno de los anteriores. Si el usuario tiene menos de tres amigos, se muestran los que existan (0, 1, 2 o 3).

5.3. Solicitudes de amistad

Las solicitudes se representan en una segunda matriz de bits. El bit `[emisor][receptor]` vale 1 cuando existe una solicitud en ese sentido. Al enviar se valida que ambos índices existan, que no sea una solicitud a sí mismo, que todavía no sean amigos y que no exista una solicitud pendiente en ninguno de los dos sentidos. Si todo es válido, se enciende el bit correspondiente.

Aceptar o rechazar una solicitud concreta se resuelve mediante acceso directo a ese bit. Al aceptar, primero se comprueba que la solicitud exista, luego se establece la amistad y finalmente se apaga el bit de la solicitud. Al rechazar solamente se apaga el bit. Para mostrar todas las solicitudes recibidas por un usuario se recorre su columna completa en la matriz, por lo que esta operación sí depende de la cantidad de usuarios.

5.4. Sugerencias de amistad

Para no llamar `contarAmigosComunes` por cada candidato, el algoritmo recorre los amigos directos del usuario y, por cada amigo, recorre todos los posibles candidatos. Si ese amigo está conectado con un candidato que todavía no es amigo del usuario, se suma uno a la puntuación

de ese candidato. Al final, la puntuación equivale a la cantidad de amigos en común. El propio usuario, sus amigos y quienes quedan con 0 amigos en común no aparecen.

Después se hace un recorrido adicional para eliminar de las sugerencias a los usuarios que ya tienen una solicitud pendiente con el usuario actual, sin importar en cuál de los dos sentidos fue enviada.

Para mostrar las K sugerencias no ordenamos todo el arreglo, porque solo necesitamos los K mejores y K no pasa de 20. En su lugar repetimos hasta K veces lo mismo: recorremos las puntuaciones para encontrar al candidato con más amigos en común, lo agregamos al resultado y marcamos su puntuación con -1 para que no vuelva a salir. En caso de empate gana el menor índice, porque se recorre desde 0 y solo se reemplaza al mejor cuando aparece una puntuación estrictamente mayor. Si hay menos de K candidatos válidos, se devuelven únicamente los que existan.

5.5. Mensajes

Enviar. Se comprueba que emisor y receptor existan y sean amigos. Luego se valida que el texto no sea nulo, que no esté vacío y que no pase de 512 caracteres, y también se valida la fecha. El nuevo mensaje se escribe directamente en la siguiente posición disponible del último bloque de 64. Después se enlaza el mismo objeto al final de la lista de enviados del emisor y de la lista de recibidos del receptor.

Leer nuevos. Se recorre la lista enlazada de mensajes recibidos del usuario y se seleccionan los que todavía no han sido leídos. Los punteros a esos mensajes se guardan en un arreglo temporal y se ordenan por fecha mediante inserción. Luego se muestran en ese orden y cada mensaje mostrado se marca como leído.

Buscar por rango de fechas. Si la fecha final es menor que la inicial, el rango se considera inválido y el menú muestra el aviso correspondiente. En caso contrario se recorre la lista de recibidos, se seleccionan únicamente los mensajes cuya fecha está dentro del intervalo inclusivo y se ordenan temporalmente por fecha antes de mostrarlos.

Este procedimiento hace que el resultado sea cronológico aunque el archivo o el orden de llegada no lo sean. Los objetos `Mensaje` no se copian durante el ordenamiento: solo se reorganiza un arreglo temporal de punteros.

5.6. Perfil de un amigo y carrusel

Un álbum se muestra según su privacidad: el propietario siempre puede verlo; si es público, cualquier visitante puede verlo si es de amigos, solo se muestra cuando visitante y dueño son amigos; y si es privado, nadie más que el propietario puede verlo.

El carrusel es circular: para pasar a la siguiente foto se calcula $(\text{actual} + 1)$ mód total y para la anterior $(\text{actual} - 1 + \text{total})$ mód total, de modo que desde la última foto se vuelve a la primera y viceversa. Durante la visita, un arreglo temporal permite alternar entre dar y quitar *like*. El *like* modifica la foto real del álbum, así que se conserva en memoria al salir del carrusel y se guarda en `fotos.txt` al cerrar sesión.

6. Medición del consumo de recursos

Al terminar cada funcionalidad se muestran las métricas administradas por la clase `Metricas`: iteraciones, memoria total en ese momento y componentes externos utilizados.

Iteraciones. Contamos como iteración cada vuelta de los ciclos que instrumentamos explícitamente con `Metricas`. Por ejemplo, `contarAmigosComunes` suma una iteración por cada usuario revisado (1000 con 1000 usuarios), mientras que consultar si dos usuarios son amigos es un acceso directo a un bit. Las cargas, guardados, selección de sugerencias, recorridos de mensajes y ordenamientos también incrementan el contador en los ciclos instrumentados.

Las operaciones realizadas internamente por componentes de la biblioteca estándar no se expanden como iteraciones propias. En su lugar se reporta que se utilizó ese componente externo.

Memoria permanente. `getMemoriaSistema` parte de `sizeof(UdeABook)` y suma la matriz triangular de amistades y la matriz completa de solicitudes, el arreglo completo de usuarios y la memoria dinámica de cada usuario, incluyendo sus cadenas, álbumes, fotos y rutas. También se cuentan los bloques completos reservados para `Mensaje`, los textos de los mensajes y la cadena utilizada para conservar la ruta de la carpeta de datos. Los punteros de las listas de enviados y recibidos ya están incluidos dentro del tamaño de `Usuario` y `Mensaje`; los objetos mensaje se cuentan una sola vez, dentro de `UdeABook`.

Memoria temporal. En las funcionalidades instrumentadas se suma la memoria de variables locales, parámetros por valor y arreglos temporales cuando corresponde, y se resta cuando dejan de estar activos. Al terminar una funcionalidad, `registrarMemoriaFinal` calcula la memoria reportada como la memoria permanente del sistema más la memoria temporal que continúa activa en ese momento. No se calcula un máximo histórico o pico: la medición corresponde al momento exacto en que se reporta la funcionalidad.

Componentes externos. La implementación registra de forma separada el uso de `<random>` para la selección aleatoria, `<chrono>` para obtener la fecha actual, `<fstream>` para las operaciones con archivos. Se reporta cuántas veces se invocó cada tipo de componente, sin contar sus ciclos internos como iteraciones propias del algoritmo.

7. Problemas de desarrollo

- **Propiedad de la memoria.** Más que errores finales, tuvimos que resolver varios riesgos: copias superficiales que podían terminar en doble `delete`, decidir quién era dueño de los mensajes y evitar punteros a memoria liberada. La solución final deja a `UdeABook` como único dueño de los mensajes; los usuarios solamente mantienen referencias no propietarias.
- **Dos formas de enlazar el mismo mensaje.** Un mensaje debe aparecer en los enviados del emisor y en los recibidos del receptor sin duplicarse. Para resolverlo, cada objeto tiene un enlace `siguienteEnviado` y otro `siguienteRecibido`.
- **Memoria de las relaciones.** Una matriz normal de enteros hubiera usado demasiada memoria con 1000 usuarios. Primero comprimimos amistades y solicitudes en bits. Después aprovechamos la simetría de las amistades para representarlas con una matriz triangular que guarda cada pareja una sola vez. Las solicitudes conservaron la matriz completa por ser dirigidas.
- **Carga sin copias innecesarias.** En una versión anterior se construían arreglos temporales de álbumes y fotos para luego copiarlos. La solución final reserva los arreglos definitivos con el tamaño conocido y llena cada objeto directamente en su posición.
- **Orden cronológico de mensajes.** En lugar de depender del orden del archivo, se seleccionan punteros a los mensajes que deben mostrarse y se ordenan temporalmente por fecha. Así se mantiene una sola copia de cada mensaje.
- **Operaciones costosas.** Lo que todavía puede ser costoso son las sugerencias, los amigos en común y el ordenamiento por inserción cuando un usuario tiene muchos mensajes seleccionados.

- **Medición.** Definir qué cuenta como iteración y qué memoria permanece activa al terminar una funcionalidad fue más difícil de lo esperado.

7.1. Aprendizajes

Aprendimos que la programación orientada a objetos no es solo crear clases, sino decidir responsabilidades y relaciones: quién crea, quién usa y quién destruye cada objeto. Con la memoria dinámica entendimos la importancia de los destructores, las copias profundas de los recursos que sí pertenecen a una clase, el constructor de copia, el operador de asignación y evitar dobles `delete`. También vimos que la estructura de datos que se escoge cambia mucho el consumo de memoria y el rendimiento del programa. La compresión triangular de amistades fue un ejemplo concreto: al aprovechar una propiedad del problema, redujimos memoria sin perder el acceso directo a las relaciones.